Tribhuvan University

Faculty of Humanities and Social Sciences

"Cloud Computing"

A LAB REPORT

Submitted To:

Department of Computer Application

Kathmandu College of Central State

Soalteemode, Kathmandu

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by:

Pramod Gole (6-2-528-12-2021)

Under Supervision of

Mr. Bishranta Bhattarai

TABLE OF CONTENTS

Lab 1: Virtualization 1

1. Creating a virtual machine 2
2. Accessing VM from host OS..... 4
3. Accessing VM from host OS..... 5
5. Accessing network between VMs 8
6. Internal network implementation 9
7. Bridged Network Mode 10

Lab 2: Map and Reduce 12

Lab 1: Virtualization

Introduction:

Virtualization is the technique of creating a virtual version of computing resources such as servers, storage devices, networks, or operating systems. Instead of relying on separate physical hardware for each system, virtualization enables multiple virtual environments to operate on a single physical machine. This improves resource utilization, flexibility, and cost efficiency.

Hypervisor:

A hypervisor is a special software (or firmware/hardware layer) that creates and manages virtual machines (VMs). It allows multiple operating systems to run on a single physical machine by sharing its hardware resources like CPU, memory, and storage.

Types of Hypervisors

1. **Type 1 (Bare-Metal Hypervisor)**
   - Installed directly on physical hardware
   - Provides higher performance and better security.

   Examples: VMware ESXi, Microsoft Hyper-V, Xen, KVM

2. **Type 2 (Hosted Hypervisor)**
   - Runs as software on top of an existing operating system
   - Easier for personal and development use

   Examples: VMware Workstation, Oracle VirtualBox, Parallels Desktop

Types of Networks:

1. NAT (Network Address Translation)

NAT is the default networking mode in VirtualBox. In this mode, the virtual machine uses the host computer's IP address to access external networks like the internet. The VM remains hidden from the local network, meaning other devices cannot access it directly. This provides good security and is ideal for general internet usage without requiring special network configuration.

2. Bridged Adapter

In Bridged networking, the virtual machine connects directly to the physical network through the host's network adapter. The VM receives its own IP address from the network's DHCP server, just like another computer connected to the same LAN. This

allows full communication with other devices on the network and is suitable for server deployment, testing, and accessing shared network resources.

3. Host-Only Adapter

Host-Only networking creates a private network between the virtual machine and the host system only. The VM cannot access the internet or other devices on the LAN unless additional routing is configured. This isolated setup is useful for testing, secure communication, and file sharing between the host and virtual machine without exposing the VM to the outside network.

4. Internal Network

Internal Network mode isolates the VM completely from the host and external networks. Only virtual machines connected to the same internal network can communicate with each other. This network type is ideal for environments that require high security or controlled testing conditions, such as experimenting with network protocols or malware analysis.

5. NAT Network

NAT Network works similarly to NAT but with one major difference: multiple VMs can communicate with each other while still accessing the internet through the host. Each VM receives a private IP address inside the NAT network. This mode is useful for multi-VM labs where communication between virtual machines and external access are both required.

1. Creating a virtual machine

Creating a virtual machine typically involves:

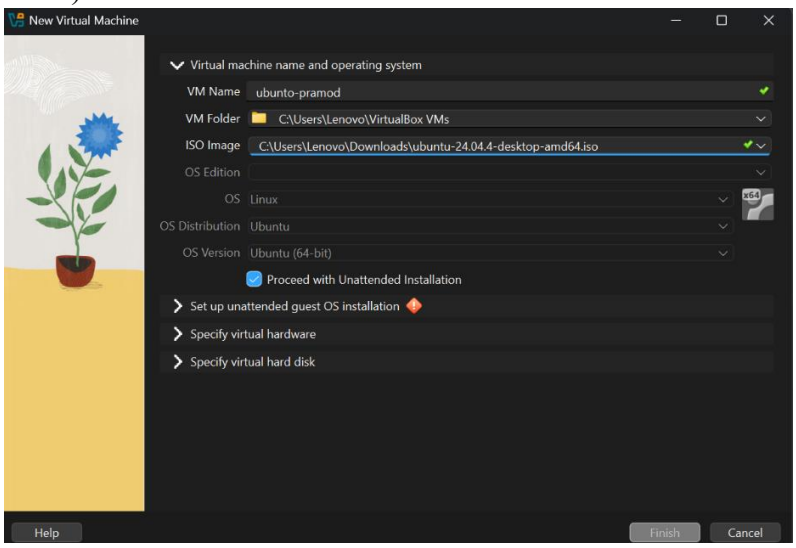
- Selecting a hypervisor (e.g., VMware, VirtualBox, Hyper-V, KVM) to manage and allocate resources.
- Allocating resources such as CPU, memory, and storage based on requirements.
- Installing an operating system inside the VM to make it function like an independent computer.
- Configuring networking so the VM can communicate with other systems or the internet.

Steps involved:

Step 1: Open VirtualBox.

Step 2: Click New.

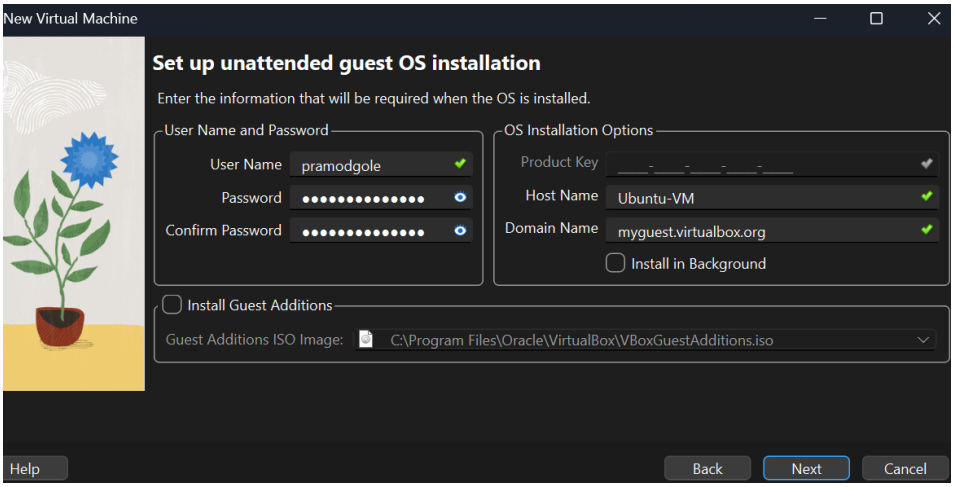
Step 3: Enter the name of your VM. VirtualBox usually auto-selects the type (Linux/Windows) and version based on the name.



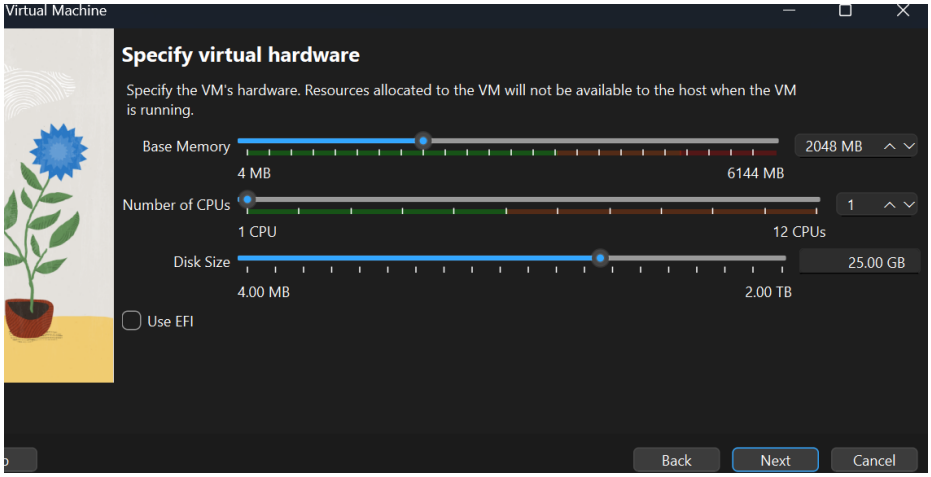
Step 4: Choose the Machine Folder (where VM files will be saved).

Step 5: Select the ISO Image file or set the path of the iso file.

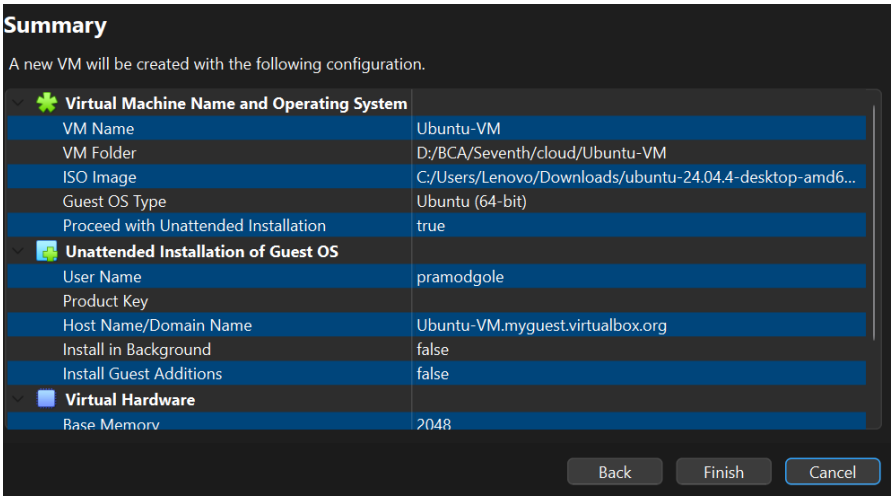
Step 6: Set the username and password for the VM.



Step 7: Select how much RAM to allocate and make sure that it stays within the green zone as well as for the CPUs also.



Step 8: Now click on Finish button.



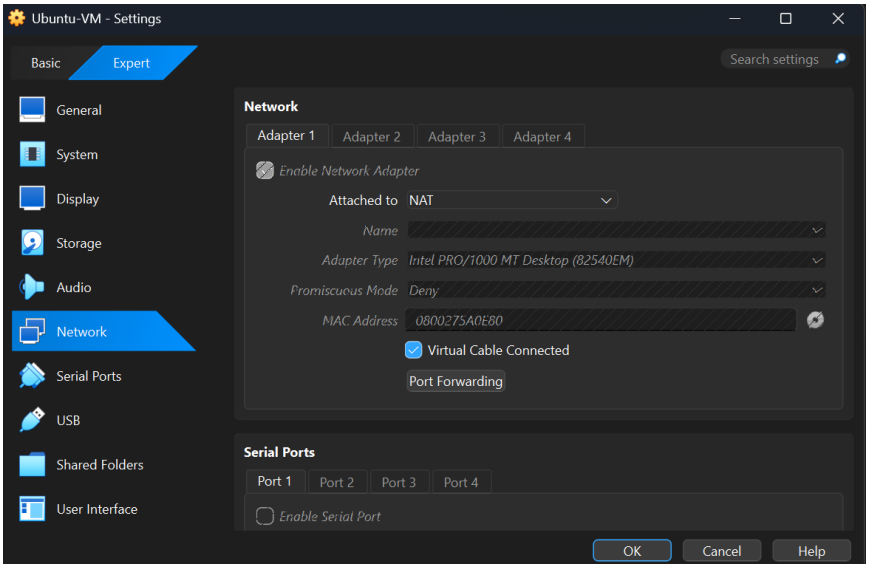
2. Accessing VM from host OS

Introduction:

After creating a virtual machine, the next step is to access and interact with it from the host operating system. Accessing a VM allows the user to control its desktop, run applications, and manage files just like on a physical computer. There are several ways to achieve this, depending on the virtualization platform and network configuration. One of them is Network-Based Access, if networking is configured (e.g., bridged or NAT mode), users can connect to the VM from the host using protocols like SSH (for Linux VMs) or Remote Desktop (for Windows VMs).

Steps Involved:

Step 1: Open the VM settings in VirtualBox and go to the Network section. Make sure the network is set to NAT instead of Bridged Adapter.



Step 2: Start the virtual machine and check its IP address. Type ip a in the terminal to know the IP address.

```
pramodgole@ubunto-pramod: ~/Desktop
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

pramodgole@ubunto-pramod:~/Desktop$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:50:fb brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86224sec preferred_lft 86224sec
    inet6 fd17:625c:f037:2:f87c:bb92:2878:f040/64 scope global temporary dynamic
       valid_lft 86291sec preferred_lft 14291sec
    inet6 fd17:625c:f037:2:a00:27ff:fe60:50fb/64 scope global dynamic mngtmpaddr
       valid_lft 86291sec preferred_lft 14291sec
    inet6 fe80::a00:27ff:fe60:50fb/64 scope link
       valid_lft forever preferred_lft forever
```

Step 3: On the host computer, open the terminal and try pinging the guest OS using its IP address. If we see replies coming back, it means our host machine can successfully communicate with the virtual machine.

```
pramodgole@ubunto-pramod:~/Desktop$ ping 192.168.1.86
PING 192.168.1.86 (192.168.1.86) 56(84) bytes of data.
64 bytes from 192.168.1.86: icmp_seq=1 ttl=255 time=2.48 ms
64 bytes from 192.168.1.86: icmp_seq=2 ttl=255 time=1.15 ms
64 bytes from 192.168.1.86: icmp_seq=3 ttl=255 time=1.14 ms
64 bytes from 192.168.1.86: icmp_seq=4 ttl=255 time=0.975 ms
64 bytes from 192.168.1.86: icmp_seq=5 ttl=255 time=0.785 ms
64 bytes from 192.168.1.86: icmp_seq=6 ttl=255 time=1.22 ms
64 bytes from 192.168.1.86: icmp_seq=7 ttl=255 time=1.24 ms
64 bytes from 192.168.1.86: icmp_seq=8 ttl=255 time=1.02 ms
64 bytes from 192.168.1.86: icmp_seq=9 ttl=255 time=1.16 ms
64 bytes from 192.168.1.86: icmp_seq=10 ttl=255 time=0.776 ms
64 bytes from 192.168.1.86: icmp_seq=11 ttl=255 time=1.04 ms
```

3. Accessing VM from host OS

Accessing a Virtual Machine (VM) from the host operating system is an important step that allows users to interact with the VM as if it were a separate physical computer. To access services running inside a Virtual Machine (VM) from the host system, we need to set up networking and verify connectivity.

Steps Involved:

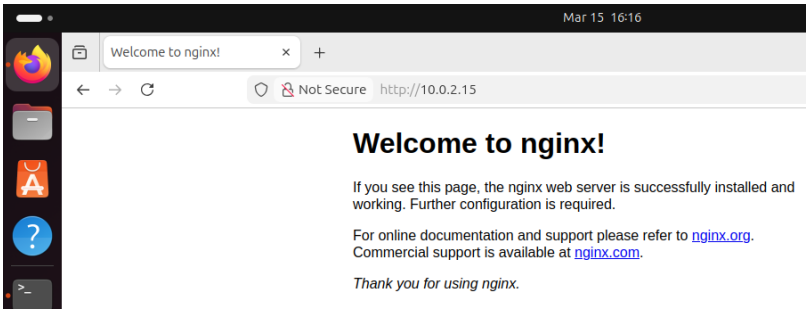
Step 1: On the guest OS, install the Nginx server using “sudo apt install nginx”.

```
pramodgole@ubunto-pramod:~/Desktop$ sudo apt install -y nginx
[sudo] password for pramodgole:
Sorry, try again.
[sudo] password for pramodgole:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 29 not upgraded.
Need to get 565 kB of archives.
After this operation, 1,596 kB of additional disk space will be used.
Get:1 http://np.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx-common all 1.24.0-2ubuntu7.6 [43.5 kB]
Get:2 http://np.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx amd64 1.24.0-2ubuntu7.6 [521 kB]
Fetched 565 kB in 6s (99.8 kB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 151263 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.24.0-2ubuntu7.6_all.deb ...
Unpacking nginx-common (1.24.0-2ubuntu7.6) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.24.0-2ubuntu7.6_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.6) ...
Setting up nginx-common (1.24.0-2ubuntu7.6) ...
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr
/lib/systemd/system/nginx.service.
Setting up nginx (1.24.0-2ubuntu7.6) ...
 * Upgrading binary nginx                                            [ OK ]
```

```
Processing triggers for ufw (0.36.2-6) ...
pramodgole@ubunto-pramod:~/Desktop$ sudo systemctl enable --now nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
pramodgole@ubunto-pramod:~/Desktop$ sudo systemctl start nginx
pramodgole@ubunto-pramod:~/Desktop$
```

Step 2: Find the IP address of the guest OS using "ip a".

Step 3: On the host machine, open a web browser and enter http://<guest_ip>, if everything is setup correctly, we can see the Nginx welcome page as shown below.



4. Accessing guest OS in VM using SSH

Introduction:

Secure Shell (SSH) is a protocol used to securely connect and manage remote systems over a network. Instead of working directly on the VM console, SSH allows the host machine to establish a secure terminal session with the guest operating system. This is especially useful for system administration, remote configuration, and file transfers. In virtualization, SSH is often used to manage Linux-based guest OS instances from the host.

To make this possible, we must ensure that the SSH service is running inside the VM and that the network settings are properly configured to allow the host to connect. In VirtualBox, this is commonly achieved using NAT with port forwarding, where a port on the host maps to the SSH port (22) on the guest.

Steps Involved:

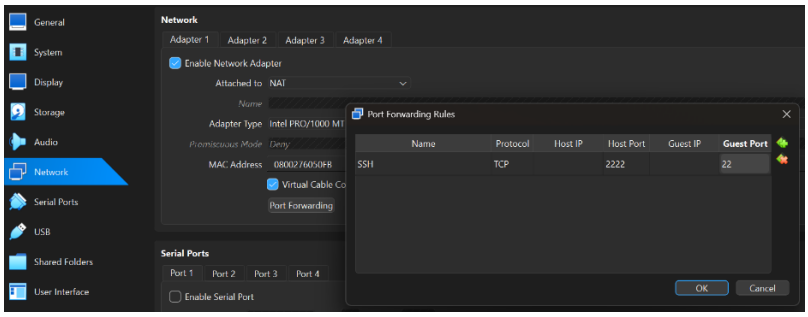
Step 1: On the guest OS, install and start the SSH server. For a Linux guest, open the terminal and run sudo apt install -y OpenSSH-server followed by sudo systemctl enable --now ssh to start the service.

```
pramodgole@ubunto-pramod:~$ sudo apt install openssh-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ncurses-term openssh-sftp-server ssh-import-id
Suggested packages:
  molly-guard monkeysphere ssh-askpass
The following NEW packages will be installed:
  ncurses-term openssh-server openssh-sftp-server ssh-import-id
0 upgraded, 4 newly installed, 0 to remove and 29 not upgraded.
Need to get 832 kB of archives.
After this operation, 6,743 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://np.archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-sftp-server amd64 1:9.6p1-3ubuntu13.15 [37.3
kB]
Get:2 http://np.archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-server amd64 1:9.6p1-3ubuntu13.15 [510 kB]
Get:3 http://np.archive.ubuntu.com/ubuntu noble/main amd64 ncurses-term all 6.4+20240113-1ubuntu2 [275 kB]
Get:4 http://np.archive.ubuntu.com/ubuntu noble-updates/main amd64 ssh-import-id all 5.11-0ubuntu2.24.04.1 [10.1 kB]
Fetched 832 kB in 3s (307 kB/s)
Preconfiguring packages ...
Selecting previously unselected package openssh-sftp-server.
(Reading database ... 151310 files and directories currently installed.)
```

```
pramodgole@ubunto-pramod:~$ sudo systemctl enable -now ssh
Failed to parse lines 'ow'
pramodgole@ubunto-pramod:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/sshd.service → /usr/lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /usr/lib/systemd/system/ssh.service.
pramodgole@ubunto-pramod:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-03-15 16:32:19 UTC; 51s ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 9106 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 9108 (sshd)
      Tasks: 1 (limit: 3452)
     Memory: 1.2M (peak: 1.4M)
        CPU: 29ms
     CGroup: /system.slice/ssh.service
             └─9108 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Mar 15 16:32:19 ubunto-pramod systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Mar 15 16:32:19 ubunto-pramod sshd[9108]: Server listening on 0.0.0.0 port 22.
Mar 15 16:32:19 ubunto-pramod sshd[9108]: Server listening on :: port 22.
Mar 15 16:32:19 ubunto-pramod systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
pramodgole@ubunto-pramod:~$
```

Step 2: In VirtualBox, make sure the VM's network is set to NAT. Then, go to Advanced → Port
Forwarding and add a rule that forwards a host port, for example 2222, to the guest's SSH port 22.



Step 3: On your host machine, open PuTTY. In the host name field, enter 127.0.0.1 and set the port to 2222, making sure SSH is selected as the connection type.

Step 4: Click Open, and enter the guest OS username and password. After logging in, we will have terminal access to the guest OS from the host, allowing us to manage the VM remotely just like a regular computer.

```
pramodgole@ubunto-pramo

PS C:\Users\Lenovo> ssh pramodgole@127.0.0.1 -p 2222
The authenticity of host '[127.0.0.1]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:FWmohKkCKZkI2L72jOcxh5Msrqm/hcMDNNTRODY/+oA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '[127.0.0.1]:2222' (ED25519) to the list of known hosts.
pramodgole@127.0.0.1's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-19-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

29 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

pramodgole@ubunto-pramod:~$
```

5. Accessing network between VMs

Introduction:

In virtualization, multiple guest operating systems (VMs) often need to communicate with each other just like computers in a physical network. This is achieved through the network configuration provided by the hypervisor. VirtualBox, VMware, and other platforms allow different modes of networking such as NAT, Bridged Adapter, and Internal Network.

Steps Involved:

Step 1: On the second guest OS, find its IP address using commands like ip a.

```
pramodgole@ubunto-pramod:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:50:fb brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 85950sec preferred_lft 85950sec
    inet6 fd17:625c:f037:2:bd2a:c51f:4dc3:7ecc/64 scope global temporary dynamic
       valid_lft 86272sec preferred_lft 14272sec
    inet6 fd17:625c:f037:2:a00:27ff:fe60:50fb/64 scope global dynamic mngtmpaddr
       valid_lft 86272sec preferred_lft 14272sec
    inet6 fe80::a00:27ff:fe60:50fb/64 scope link
       valid_lft forever preferred_lft forever
```

Step 2: From the first guest OS, try pinging the second guest OS using the ping command. If we receive replies, it means the first guest can access the second guest.

```
pramodgole@ubunto-pramod:~$ ping 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.038 ms
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=0.048 ms
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=0.035 ms
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=0.036 ms
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=0.040 ms
64 bytes from 10.0.2.15: icmp_seq=6 ttl=64 time=0.042 ms
```

Step 3: Similarly, find the IP address of the first guest OS.

```
pramodgole@ubunto-pramod:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:50:fb brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86157sec preferred_lft 86157sec
    inet6 fd17:625c:f037:2:ff31:5f52:bac4:ae74/64 scope global temporary dynamic
       valid_lft 86159sec preferred_lft 14159sec
    inet6 fd17:625c:f037:2:a00:27ff:fe60:50fb/64 scope global dynamic mngtmpaddr
       valid_lft 86159sec preferred_lft 14159sec
    inet6 fe80::a00:27ff:fe60:50fb/64 scope link
       valid_lft forever preferred_lft forever
```

Step 4: From the second guest OS, ping the first guest OS. If we get replies back, it confirms that the second guest OS can access the first guest OS.

```
pramodgole@ubunto-pramod:~$ ping 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.045 ms
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=0.036 ms
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=0.039 ms
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=0.047 ms
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=0.045 ms
64 bytes from 10.0.2.15: icmp_seq=6 ttl=64 time=0.045 ms
64 bytes from 10.0.2.15: icmp_seq=7 ttl=64 time=0.041 ms
64 bytes from 10.0.2.15: icmp_seq=8 ttl=64 time=0.034 ms
```

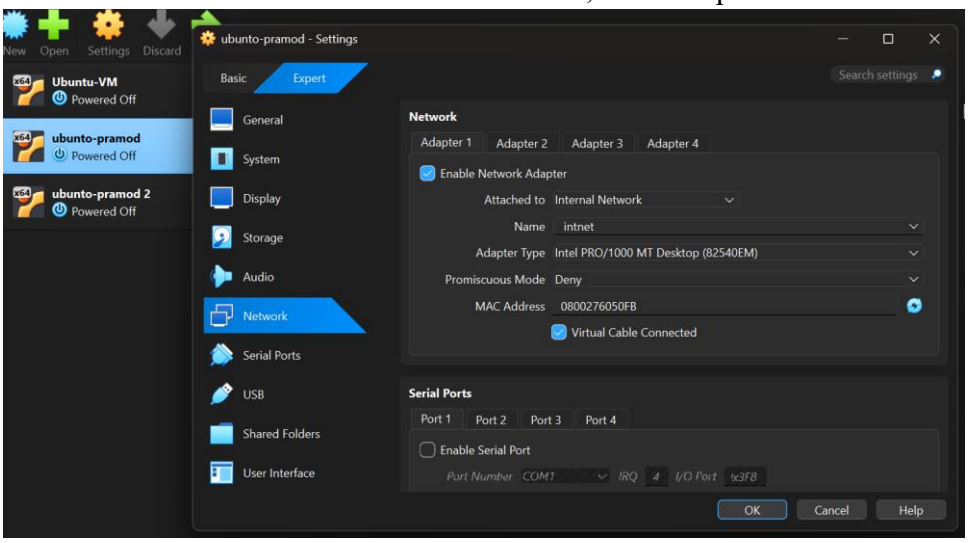
6. Internal network implementation

Introduction:

An Internal Network in virtualization allows multiple VMs to communicate with each other without involving the host machine or external network. This is particularly useful for testing client-server applications, network services, or isolated network environments in a safe and controlled setting. Each VM on the internal network can be assigned a static IP address, which ensures reliable communication across the subnet. Unlike NAT or Bridged modes, the internal network is fully isolated from the host and the internet unless additional routing is configured.

Steps Involved:

Step 1: In the settings, go to the Network tab. Select Adapter 1 and change the "NAT" option to Internal Network. Give the network a name, for example internet.



Step 3: Repeat the same process for the other VM(s) which we want to include in the internal network. Go to Settings → Network → Adapter 1 → Internal Network and enter the same network name (internet) to ensure all VMs are on the same virtual network.

Step 4: Start all the VMs that are part of the internal network. At this point, the network interfaces exist but they may not have IP addresses yet, so the VMs won't be able to communicate immediately.

Step 5: Assign static IP addresses to each VM so they can communicate on the same subnet. Open a terminal in each VM and assign an IP using commands like "sudo ip addr add 192.168.3.3/24 dev enp0s3" for the first VM and "sudo ip addr add 192.168.3.4/24 dev enp0s3" for the second VM. Replace enp0s3 with your network

interface name if it is different. Also, bring the interface up using sudo ip link set enp0s3 up.

```
pramodgole@ubunto-pramod:~$ sudo ip addr add 192.168.3.3/24 dev enp0s3
[sudo] password for pramodgole:
pramodgole@ubunto-pramod:~$ sudo ip link set enp0s3 up
pramodgole@ubunto-pramod:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:50:fb brd ff:ff:ff:ff:ff:ff
    inet 192.168.3.3/24 scope global enp0s3
       valid_lft forever preferred_lft forever
pramodgole@ubunto-pramod:~$
```

```
pramodgole@ubunto-pramod:~$ sudo ip addr add 192.168.3.4/24 dev enp0s3
[sudo] password for pramodgole:
pramodgole@ubunto-pramod:~$ sudo ip link set enp0s3 up
pramodgole@ubunto-pramod:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:50:fb brd ff:ff:ff:ff:ff:ff
    inet 192.168.3.4/24 scope global enp0s3
       valid_lft forever preferred_lft forever
pramodgole@ubunto-pramod:~$
```

Step 6: Verify the IP addresses by running "ip a" on each VM. You should see the assigned IP addresses listed for the network interfaces connected to the internal network.

Step 7: Now test connectivity between the VMs by pinging one VM from the other. If you receive replies, the internal network is set up correctly.

```
pramodgole@ubunto-pramod:~$ ping 192.168.3.4
PING 192.168.3.4 (192.168.3.4) 56(84) bytes of data.
64 bytes from 192.168.3.4: icmp_seq=1 ttl=64 time=23.1 ms
64 bytes from 192.168.3.4: icmp_seq=2 ttl=64 time=4.04 ms
64 bytes from 192.168.3.4: icmp_seq=3 ttl=64 time=1.44 ms
64 bytes from 192.168.3.4: icmp_seq=4 ttl=64 time=0.418 ms
64 bytes from 192.168.3.4: icmp_seq=5 ttl=64 time=0.484 ms
64 bytes from 192.168.3.4: icmp_seq=6 ttl=64 time=0.436 ms
64 bytes from 192.168.3.4: icmp_seq=7 ttl=64 time=0.305 ms
64 bytes from 192.168.3.4: icmp_seq=8 ttl=64 time=0.370 ms
```

```
pramodgole@ubunto-pramod:~$ ping 192.168.3.3
PING 192.168.3.3 (192.168.3.3) 56(84) bytes of data.
64 bytes from 192.168.3.3: icmp_seq=1 ttl=64 time=6.21 ms
64 bytes from 192.168.3.3: icmp_seq=2 ttl=64 time=0.426 ms
64 bytes from 192.168.3.3: icmp_seq=3 ttl=64 time=0.525 ms
64 bytes from 192.168.3.3: icmp_seq=4 ttl=64 time=0.336 ms
64 bytes from 192.168.3.3: icmp_seq=5 ttl=64 time=0.464 ms
64 bytes from 192.168.3.3: icmp_seq=6 ttl=64 time=0.437 ms
64 bytes from 192.168.3.3: icmp_seq=7 ttl=64 time=0.388 ms
```

7. Bridged Network Mode

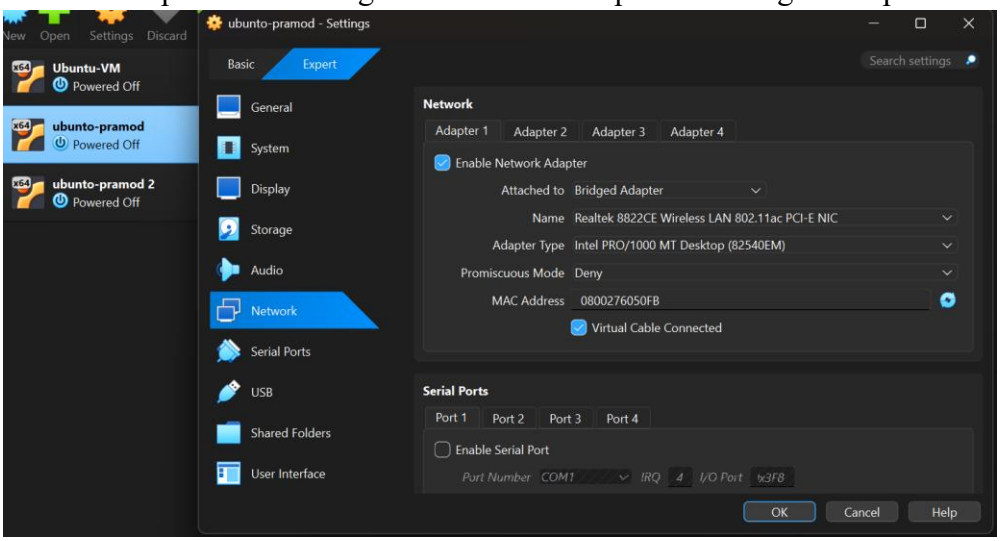
Introduction:

Bridged Network Mode allows a virtual machine (VM) to connect directly to the same physical network as the host machine. In this mode, the VM behaves like a separate physical computer on the network, receiving its own IP address from the network's DHCP server (or can be assigned a static IP).

Steps Involved:

Step 1: Open the VM settings and go to the Network tab.

Step 2: Select Adapter 1 and change the Attached to option to Bridged Adapter.

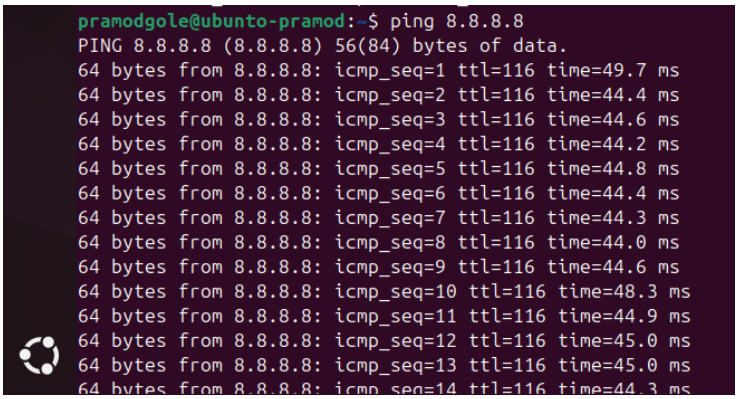


Step 3: Choose the physical network interface of your host machine that is connected to the LAN (e.g., Ethernet or Wi-Fi adapter).

Step 4: Start the VM. The VM will request an IP address from the network's DHCP server. You can verify the IP using: ip a # For Linux and ipconfig # For Windows.

```
pramodgole@ubunto-pramod:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:50:fb brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.64/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86273sec preferred_lft 86273sec
    inet6 2400:74e0:0:84a2:28cd:ab78:451e:6918/64 scope global temporary dynamic
       valid_lft 474sec preferred_lft 474sec
    inet6 2400:74e0:0:84a2:a00:27ff:fe60:50fb/64 scope global dynamic mngtmpaddr
       valid_lft 474sec preferred_lft 474sec
    inet6 fe80::a00:27ff:fe60:50fb/64 scope link
       valid_lft forever preferred_lft forever
```

Step 5: Test connectivity by pinging other devices on the same network, including the host machine, other VMs, or network servers.



```
pramodgole@ubunto-pramod:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=49.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=44.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=44.6 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=116 time=44.2 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=116 time=44.8 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=116 time=44.4 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=116 time=44.3 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=116 time=44.0 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=116 time=44.6 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=116 time=48.3 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=116 time=44.9 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=116 time=45.0 ms
64 bytes from 8.8.8.8: icmp_seq=13 ttl=116 time=45.0 ms
64 bytes from 8.8.8.8: icmp_seq=14 ttl=116 time=44.3 ms
```

Lab 2: Map and Reduce

Map:

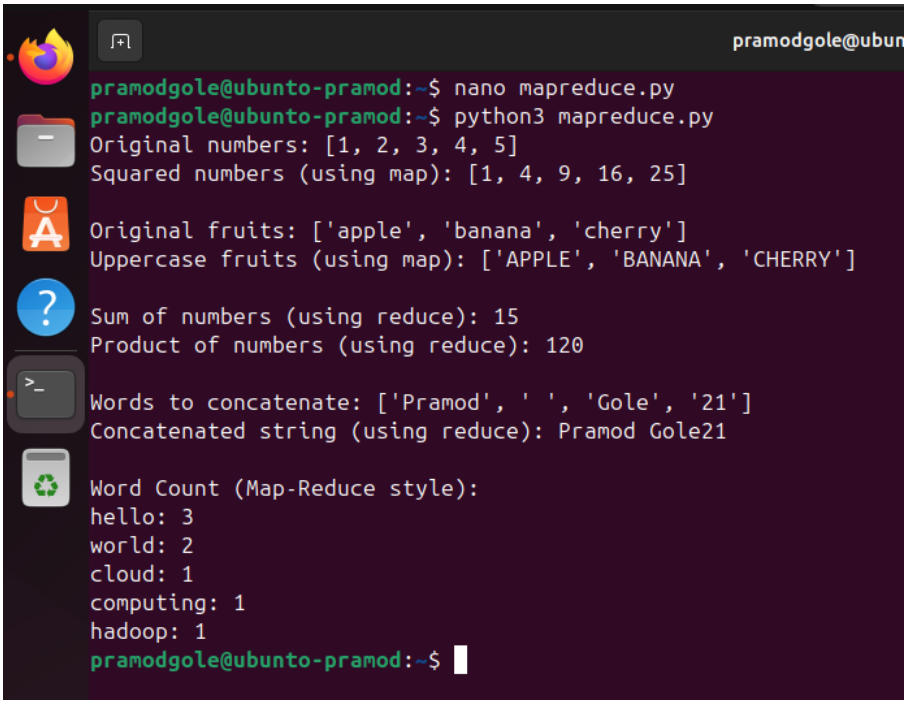
In cloud computing, the Map function is used to process large datasets by dividing the input data into smaller chunks and distributing them across multiple compute nodes. Each node performs the same operation independently on its assigned data. This parallel processing approach speeds up data handling and increases scalability. Map is commonly used in cloud environments for tasks like filtering, data transformation, and extraction in big data applications.

Reduce:

The Reduce function in cloud computing is used to combine the processed outputs generated by the Map function into a single aggregated result. It collects intermediate values from distributed nodes and performs operations such as summation, counting, or merging. Reduce ensures efficient consolidation of large-scale data, making it vital for analytics, reporting, and big data processing in cloud platforms.

Code:

```
from functools import reduce
# ---------------------------
# Map Examples
# ---------------------------
# 1. Squaring numbers
numbers = [1, 2, 3, 4, 5]
squared_numbers = list(map(lambda x: x * x,
numbers))
print(f"Original numbers: {numbers}")
print(f"Squared numbers (using map):
{squared_numbers}\n")
# 2. Uppercasing fruits
fruits = ["apple", "banana", "cherry"]
uppercase_fruits = list(map(str.upper, fruits))
print(f"Original fruits: {fruits}")
print(f"Uppercase fruits (using map):
{uppercase_fruits}\n")
# ---------------------------
# Reduce Examples
# ---------------------------
# 1. Sum and product of numbers
sum_of_numbers = reduce(lambda x, y: x + y,
numbers)
product_of_numbers = reduce(lambda x, y: x * y,
numbers)
```

```
print(f"Sum of numbers (using reduce):
{sum_of_numbers}")
print(f"Product of numbers (using reduce):
{product_of_numbers}\n")
# 2. Concatenate words into a single string
words = ["Pramod", " ", "Gole", "21"]
concatenated_string = reduce(lambda x, y: x + y,
words)
print(f"Words to concatenate: {words}")
print(f"Concatenated string (using reduce):
{concatenated_string}\n")
# ---------------------------
# Word Count Example (Map-Reduce style)
# ---------------------------
text_list = ["hello world", "hello", "cloud
computing", "hello world hadoop"]
# Split each sentence into words (map)
words_flat = [word for sentence in text_list for
word in sentence.split()]
# Count occurrences (reduce)
word_count = {}
for word in words_flat:
    word_count[word] = word_count.get(word, 0)
+ 1
print("Word Count (Map-Reduce style):")
for word, count in word_count.items():
    print(f"{word}: {count}")
```

Output: